

Lab 4: Single-Core and Multi-Core Systems

1 Introduction

The purpose of this lab was to integrate previously developed high-performance microarchitectural components into complete computer systems and evaluate how performance and complexity scale when transitioning from a single-core to a multicore architecture. In our earlier labs we produced optimized components including a variable-latency multiplier (Lab 1), a five-stage pipelined TinyRV2 processor with bypassing to avoid data hazards (Lab 2), and a two-way set-associative cache that reduces conflict misses and improves memory performance (Lab 3). In this lab, we composed these elements into fully functional RISC-V systems capable of running compiled C applications, reinforcing key concepts from lecture about hardware and software co-design, thread-level parallelism, and shared-memory performance behavior. The baseline system consists of a single TinyRV2 processor with private instruction and data caches running a single-threaded sorting microbenchmark. The alternative system scales this design to four cores, each with a private instruction cache and a shared four-bank data cache accessed through a custom ring network. Additionally, we developed a multi-threaded sorting microbenchmark using the `bthread` library, where each core sorts part of the array before core 0 performs a final merge. Directed and randomized unit tests verified the ring network, arbitration logic, and shared banked data cache. Functional-level and RTL C-based tests validated the correctness of the single-threaded and multi-threaded sorting software. We evaluated system performance using five microbenchmarks (`vvadd`, `sort`, `mfilt`, `cmult`, and `bsearch`), using cycle counts, CPI measurements, and normalized speedup metrics to quantify the benefits and overheads of multicore execution. When thread-level parallelism was effectively exploited, the multicore system delivered significant gains. For example, `mfilt` achieved a $2.27\times$ speedup and `bsearch` reached $2.38\times$, benefiting from reduced stalls, lower CPI, and increased memory concurrency. In contrast, `vvadd` and `sort` showed more modest gains of $1.64\times$ and $1.51\times$ due to memory bandwidth pressure and synchronization during the merge phase. Although the multicore system consistently reduced CPI, no benchmark approached the ideal $4\times$ speedup because thread setup, load imbalance, and shared-cache contention introduce additional delays. Single-threaded execution on the multicore hardware performed worse than the baseline because added control and arbitration complexity increased energy use without improving performance. Overall, multicore acceleration delivers strong improvements only when parallel work dominates and inter-core coordination is minimal; otherwise, added hardware cost yields diminishing returns.

2 Alternative Design

In contrast to the single-core baseline, our alternative design instantiates four TinyRV2 cores, each with a private instruction cache, and a shared four-banked data cache connected through a pair of on-chip ring networks. This design exploits thread-level parallelism (TLP) by running multiple software threads on different cores while using banking and networking to increase effective memory bandwidth. The overall block diagram (Figure 1) follows the multi-core organization described in the lab handout: four processors, four private instruction caches, a shared banked data cache, and separate request and response networks between processors, cache banks, and main memory.

2.1 Multi-Core Hardware

2.1.1 Ring Network and Router Microarchitecture

The interconnect between cores, cache banks, and memory is a ring network of four identical routers and four terminal interfaces (one per core or cache bank). Each router has three input and three output streams: local, clockwise, and counter-clockwise. Messages carry a source field, destination field, opaque field (for higher-level bookkeeping), and a payload containing the memory request or response.

Internally, each router consists of input queues, a route unit, and switch units. Incoming messages are first buffered, then the route unit uses the router ID and destination field to choose an output using a shortest-path routing algorithm. If the destination terminal is local, the message is sent to the local output; otherwise,

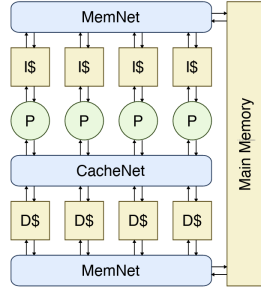


Figure 1: Overview of the multi-core system with four TinyRV2 cores, private instruction caches, and a shared banked data cache connected via on-chip networks.

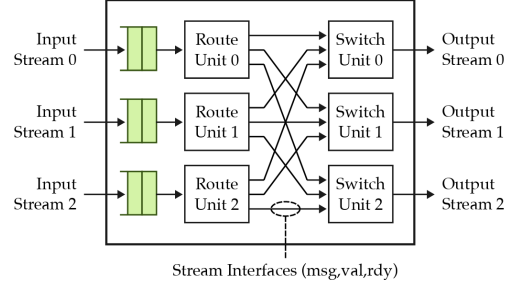


Figure 2: Router micro-architecture with input queues, route units, and switch units implementing shortest-path routing and round-robin arbitration.

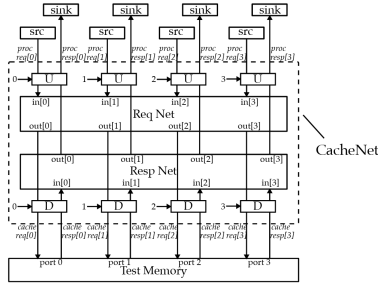


Figure 3: CacheNet constructed from request and response rings with adapters.

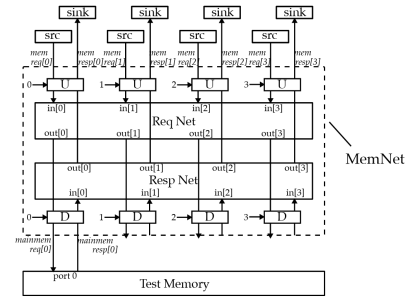


Figure 4: MemNet constructed from request and response rings with adapters.

the route unit compares clockwise and counter-clockwise hop distances and selects the shorter path, breaking ties deterministically (e.g., always clockwise) to reduce average hop count and network latency.

After routing, the switch unit arbitrates among messages targeting the same output. It uses round-robin arbitration with a rotating pointer so that competing inputs take turns accessing each output, providing fairness and avoiding starvation that can occur with fixed-priority schemes. Flow control throughout the ring uses the standard `msg/val/rdy` handshake. If a downstream component deasserts ready, the corresponding input queue fills and back-pressure propagates upstream, naturally stalling route and switch logic and managing congestion without extra control protocols.

2.1.2 Cache and Memory Networks (CacheNet and MemNet)

On top of the basic ring routers, the multi-core design uses two logical networks to connect the cores, data cache banks, and main memory. The first, *CacheNet*, interconnects the four processors with the four data cache banks, while the second, *MemNet*, connects the four data cache banks to the single main memory port. Each logical network is implemented as two physical ring networks: one dedicated to request messages and one to response messages.

At every terminal, a pair of adapters translates between memory-system messages and network messages. The upstream adapter (on the requester side) converts memory requests into network messages by setting the source field to the terminal ID (core or bank), deriving the destination from the address's bank bits or a fixed main-memory ID, and copying the payload and opaque fields so responses can be matched to requests. The downstream adapter (on the responder side) performs the inverse, reconstructing memory response messages from the network payload and opaque fields and using the source/destination information to route the response back to the correct requester.

This adapter-based structure cleanly decouples the memory protocol from the network implementation.

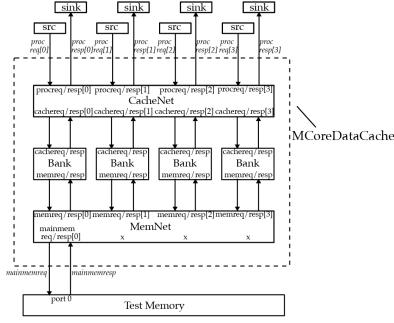


Figure 5: Multi-core data cache composed of four banks, a cache network, and a memory network.

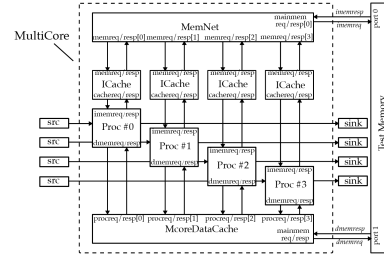


Figure 6: Complete four-core MultiCore module.

The routers are agnostic to the semantics of the messages they carry; they only interpret the source and destination fields when making routing decisions. This decoupling allows the same ring router design to be reused across both CacheNet and MemNet, reducing design complexity and verification effort.

2.1.3 Multi-Core Data Cache and MultiCoreSys

The multi-core data cache combines the cache network, four banked data cache instances, and the memory network into a single shared module. The cache network carries requests and responses between the four processors and the four data cache banks, each of which is a copy of the blocking cache from the previous lab. The memory network connects these banks to the single main memory port. Bank selection is based on bank bits in the physical address, so different lines map to different banks. When threads access data that is well-distributed across banks, cores can access different banks in parallel, increasing effective memory throughput; when they contend for the same bank, accesses are serialized by the banked cache and underlying network.

The **MultiCoreSys** top level instantiates four TinyRV2 processors, four private (unbanked) instruction caches, and one multi-core data cache. Each core keeps its own `mngr2proc/proc2mngr` interfaces for testing, but all cores share a unified physical address space via the banked data cache and networks. Core IDs are wired into the cache network adapters and used as the source field in network messages; the same IDs are visible in software through CSRs so multi-threaded code can distinguish which core it is running on. This design adds complexity in routers, networks, and banking, but enables true multi-threaded execution with multiple outstanding memory accesses in flight.

2.2 Multi-Threaded Sorting Software (mtbmark_sort)

To exploit the additional cores, we implemented a multi-threaded sorting benchmark, `mtbmark_sort`, using the `bthread` library. Its goal is to perform the same task as the single-threaded `ubmark_sort`—sorting an integer array—while reducing execution time by leveraging multiple cores and reusing the same $O(N \log N)$ algorithm.

Our parallelization follows a partition-parallel sort-merge structure. For an array of size N , we divide the array into four contiguous subarrays of roughly equal size, assigning each to a different thread. We spawn three worker threads with `ece4750_bthread_spawn`, while the main thread (on core 0) sorts the remaining subarray. Each thread simply invokes `ubmark_sort` on its local region. After starting the local sorts, the main thread calls `ece4750_bthread_join` on each worker to ensure all chunks are fully sorted.

Once the four subarrays are sorted independently, the main thread performs a deterministic merge to produce a globally sorted array. In our implementation, we first merge chunk 0 with chunk 1 and chunk 2 with chunk 3, then merge these two intermediate results. This concentrates most of the serial work into a final merge phase while allowing the bulk of the comparisons and moves to occur in parallel.

All threads share a single address space but operate on disjoint subarrays, so the algorithm avoids explicit synchronization on shared data structures. The main interaction between threads is through the

shared multi-core memory system: as the four workers traverse their subarrays, they generate independent streams of loads and stores that the banked data cache and ring networks attempt to service in parallel. Consequently, the performance of `mtbmark_sort` depends not only on the sorting algorithm but also on how effectively the memory hierarchy handles these concurrent accesses. The final merge step is largely serial and thus contributes to the sequential fraction of the program; in our evaluation we relate the observed speedups to Amdahl's Law and discuss how this serial phase and cache/network contention limit speedup relative to an ideal $4\times$ improvement.

3 Testing Strategy

This lab uses a layered testing strategy for both hardware and software. On the hardware side, we use the Python-based infrastructure from previous labs for unit, integration, and randomized tests. On the software side, we use a C testing framework built around `ECE4750_CHECK` to validate the single-threaded and multi-threaded sorting benchmarks. The goal is to gain confidence in each component in isolation and then in the composed single-core and multi-core systems, while also exposing concurrency and memory-system bugs.

We begin by re-running the complete test suites for the Lab 1 multiplier, Lab 2 TinyRV2 processor, and Lab 3 blocking cache. Verifying that these components still pass their original tests prevents us from chasing integration bugs caused by regressions in earlier designs. Only after these baselines are green do we move on to the new network, cache, and system-level components.

For the new network microarchitecture, we adopt a bottom-up approach. We first unit-test the route unit in isolation using a Python harness that drives its inputs and checks the chosen output port. Directed tests cover combinations of router IDs and destination IDs on the four-node ring, including clockwise, counter-clockwise, and tie cases, to verify shortest-path routing and deterministic tie-breaking; randomized tests further exercise the routing logic. We then unit-test the switch unit, which arbitrates between multiple inputs contending for the same output using round-robin scheduling. Conflict scenarios where several inputs continuously request the same output check that grants rotate fairly, no input is starved, and the `val/rdy` handshake remains correct even when the downstream ready signal occasionally deasserts.

After the route and switch units work in isolation, we integrate them into full routers and then into a four-node ring network. At the router level, we inject messages on different input ports and check that they emerge at the expected outputs with the correct ordering under buffering and back-pressure. At the ring level, we run tests where terminals inject streams of messages to fixed destinations, as well as randomized scenarios where destinations and injection times vary, and we introduce artificial delays on selected routers or links. These tests verify functional correctness and help rule out livelock, deadlock, and pathological contention by stressing shortest-path routing, round-robin arbitration, and `msg/val/rdy` flow control.

We then focus on the composed cache and memory networks, the multi-core data cache, and the system top levels. We reuse the provided harnesses for CacheNet, MemNet, and the Multi-Core Data Cache, and add directed tests that stress both bank-level parallelism and serialization. Some tests have four processor-side terminals issuing loads and stores to addresses that map to different banks to confirm that parallel accesses proceed concurrently; others force all terminals to target the same bank or even the same cache line, stressing bank-level serialization, miss handling, and the end-to-end path through the ring networks. By varying access patterns and injecting small delays, we expose subtle bugs in request/response matching, routing, or handling of multiple outstanding misses.

For the single-core and multi-core systems, we reuse a broad subset of the Lab 2 assembly tests to validate the processors in the context of the new memory hierarchy. These include tests for ALU operations, memory instructions, control-flow instructions, CSR instructions, and mixed sequences. We run these on both the single-core system and on the multi-core system with only one active core to cover interactions between the pipeline, caches, and networks without yet involving concurrency. We then write multi-core assembly tests using the extended syntax to give each core its own `mngr2proc` and `proc2mngr` sequence. Some tests arrange for all four cores to repeatedly read and write different words within the same cache line, while others spread accesses across different banks; these patterns are designed to catch bugs that only manifest when multiple cores issue memory operations in parallel.

On the software side, we combine unit tests of helper functions with end-to-end tests of the sorting benchmarks. For the single-threaded `ubmark_sort`, we test key subroutines such as partitioning or merging

on small corner cases (already sorted arrays, reverse-sorted arrays, duplicates, and all-equal elements) and then on larger random arrays, using `ECE4750.CHECK` to assert that outputs are sorted and that the multiset of elements is preserved. For the multi-threaded `mtbmark.sort`, we test arrays of various sizes, including sizes that are not multiples of four, and always compare the result against `ubmark.sort` on the same input. The same test binaries are run natively on the host, then on the single-core FL simulator, and finally on the multi-core FL and RTL simulators so that any RTL-only failures can be attributed to hardware rather than software bugs.

4 Optimizations

We implemented a shortest-path routing optimization in the route unit to improve network efficiency in the multi-core system. The original design forwarded all non-local messages clockwise around the ring, which could force packets to traverse up to three hops even when the destination was only one hop in the opposite direction. This increased network traffic, router contention, and memory response latency, especially when multiple cores accessed the shared data cache concurrently.

To reduce this overhead, our optimized route unit computes both clockwise and counter-clockwise hop distances and forwards each message along the shorter path. In the tie case where the destination is the same distance in both directions, we apply a simple deterministic tie-break based on router ID to avoid nondeterministic oscillation. The change is contained entirely within the combinational routing logic and integrates cleanly with round-robin arbitration and the existing request/response networks.

While we do not quantitatively measure the impact here, this routing optimization is intended to shorten end-to-end message latency, reduce network congestion, and better utilize available memory bandwidth. These effects are consistent with the performance results, where benchmarks with high parallel memory pressure (e.g., `mfilt` and `bsearch`) see the largest improvements, as fewer unnecessary hops enable the shared banked cache and multi-core pipelines to make forward progress more efficiently.

5 Evaluation

We evaluated both the baseline (single-core) and alternative (four-core) processor systems across five diverse microbenchmarks: `vvadd`, `sort`, `mfilt`, `cmult`, and `bsearch`. Each benchmark was executed using both a single-threaded (ST) program and multi-threaded (MT) variants. In particular, MT1 maps the multi-threaded program to a single hardware core (capturing multithreading overhead without parallelism), while MT4 runs the threads across all four worker cores to exploit concurrency. These microbenchmarks stress different computational and memory access characteristics: `vvadd` represents bandwidth-bound streaming arithmetic, `sort` introduces irregular access patterns and synchronization, `mfilt` resembles convolution with high temporal locality, `cmult` showcases inner-loop compute reuse, and `bsearch` features deep branch locality and limited spatial locality. Full performance statistics for all configurations are summarized in Tables 1–5.

Performance Results. Figure 7 shows that multi-threaded execution on the alternative multicore design (`Alt + MT4`) consistently outperforms the baseline single-core system (`Base + ST`), achieving speedups ranging from $1.6\times$ (`vvadd`) to $2.4\times$ (`bsearch`). Although ideal scaling for a quad-core system is $4\times$, our results clearly demonstrate Amdahl’s Law in practice: serial code regions, thread start-up, load imbalance, and additional instruction work all limit scalability. This is visible in Table 1 where `vvadd` instruction count grows from 812 (`Base ST`) to 5055 (`Alt MT1`) due to thread-launch overhead and sub-array partitioning.

Figure 8 further illustrates that CPI decreases substantially on the multicore system. This lower CPI is achieved not by a faster pipeline but by reducing stall cycles through parallel progress. However, this CPI improvement only translates into fewer cycles when the workload exposes sufficient parallelism. `Alt + ST` results confirm that multicore hardware can *hurt* serial workloads: more pipeline stages, larger branch footprint, extra I-cache pressure, and shared data resources all contribute to worse performance than `Base + ST`. This mirrors real multicore systems where legacy, serial-heavy applications often see no benefit or even degradation when moved to multi-core CPUs.

The scalability trends illustrate bottleneck differences across benchmarks. `bsearch` achieves the highest speedup because each core operates independently on separate data regions with nearly no synchronization or communication. Additionally, D-cache miss rate drops from 57% to 27% thanks to reduced memory footprint

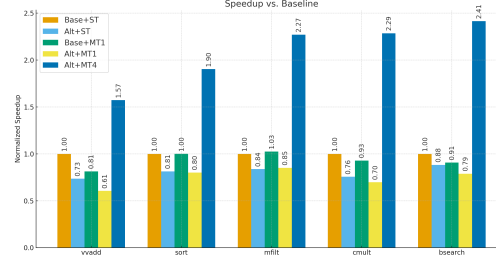


Figure 7: Execution speedup normalized to the baseline (Base ST) across all microbenchmarks and configurations. Numeric labels indicate speedup factors.

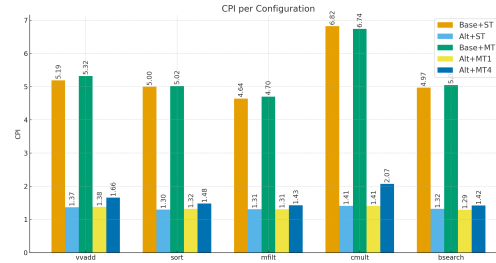


Figure 8: CPI comparison across microbenchmarks and execution configurations. Lower CPI is achieved through fewer stalls and improved pipeline utilization.

per core. Conversely, `sort` requires multiple merge phases, which serialize workload and expose both memory bandwidth and communication latency. `vadd`, though embarrassingly parallel, quickly saturates the shared memory system such that cores frequently stall waiting for D-cache line fill operations. In short: parallel compute resources cannot be fully exploited when the memory subsystem becomes the performance limiter.

Cycle Time Analysis. In our simulator, both systems share the same clock frequency, isolating architectural performance differences. In a physical implementation, the alternative system, with its deeper pipelines, ring interconnect, shared banked cache, and more complex control structures, would likely operate at a *lower* maximum clock frequency. Thus, the cycle-based speedups reported here represent best-case architectural gains. The baseline single-core design would be preferred in domains where frequency scaling and single-threaded latency dominate (e.g., microcontrollers).

Area Analysis. Area grows significantly in the alternative design: four full processor pipelines, four I-caches, a multi-banked shared D-cache, and a ring network increase logic and SRAM consumption. The `Alt + ST` case clearly wastes this additional silicon, offering worse performance per area than the baseline. Modern commercial SoCs mitigate this through aggressive power gating and asymmetric architectures (e.g., big.LITTLE), hinting that future work could dynamically disable unused cores to avoid wasted area and leakage.

Energy Analysis. Activating four cores increases dynamic power and static leakage. However, energy per task depends on work *completed per joule*, not power per cycle. When parallelism is exploited, the substantial reduction in total cycles outweighs added active hardware, as seen in `mflt`, where the cycle count is reduced from 28,951 to 12,754. For serial tasks, the multicore system is strictly worse, more power draw and longer execution. Thus, energy efficiency directly correlates with parallel speedup: workloads must expose thread-level parallelism to justify multicore power demands.

Summary and Key Takeaways. Our evaluation shows that the multicore system can achieve meaningful performance benefits, with speedups as high as 2.4x in the best cases, although it still falls short of ideal fourfold scaling because synchronization, thread management overhead, and shared memory bandwidth limitations cap parallel efficiency. While CPI improvements demonstrate that parallel execution reduces pipeline stalls when work is distributed across cores, not all workloads scale equally. Memory-bound kernels such as `vadd` see limited gains due to contention for shared bandwidth, and compute stages in `sort` require syn-

chronization that delays progress. In contrast, operations with independent subproblems like `bsearch` scale far more effectively because private working sets minimize communication between cores. Importantly, the multicore system shows negative returns when parallelism cannot be exploited, as seen in the single-threaded configuration where additional hardware only increases area and energy consumption without improving performance. Overall, these results illustrate that adding cores provides real benefit only when software exposes sufficient thread-level parallelism and when the memory subsystem can keep pace with increased concurrency. Otherwise, hardware resources are underutilized, and inefficiency becomes the dominant limiting factor.

6 Conclusion

In this lab, we integrated previously developed high-performance components into complete single-core and multicore RISC-V systems and evaluated how performance scales with parallel execution. Quantitatively, the alternative four-core design significantly improved performance for workloads with strong thread-level parallelism. The best results were observed for `bsearch` and `mfilt`, which achieved $2.38\times$ and $2.27\times$ speedups respectively over the single-core baseline, driven by lower CPI and greater memory concurrency. More modest gains of $1.64\times$ for `vvadd` and $1.51\times$ for `sort` were limited by memory bandwidth contention and serial merge phases. Importantly, no workload reached the ideal $4\times$ improvement, reaffirming Amdahl's Law: even small serial regions and synchronization overheads constrain scalability. Additionally, single-threaded execution on the multicore hardware slowed performance due to increased control complexity and shared-resource arbitration, resulting in higher energy use without benefit. Qualitatively, these results demonstrate that multicore acceleration is highly sensitive to workload characteristics. Programs that divide cleanly into independent subproblems scale well, while those bottlenecked by shared data or communication exhibit diminishing returns. For future designs, the multicore system is the superior choice only when software exposes sufficient parallelism and the memory system can sustain increased concurrency. In contrast, for workloads dominated by serial computation or frequent synchronization, the simpler single-core system delivers better performance efficiency. Extrapolating beyond our tested benchmarks, applications with larger working sets and stronger locality may benefit more from multicore scaling if bank conflicts and network contention are minimized, while highly memory-bound kernels will continue to be limited by shared bandwidth. Overall, this lab emphasizes that **adding cores alone is not enough**, architectural gains depend on balancing compute throughput with memory hierarchy capability and software design. The insights gained here form a crucial foundation for future improvements such as more advanced synchronization strategies, better load balancing, and enhancements to the shared memory subsystem.

7 Work Distribution

Both team members contributed significantly to the hardware and software development for this lab and worked closely together during debugging, integration, and final evaluation to ensure correct execution on both the single-core and multicore systems. The tasks were divided to improve the efficiency at which work was completed while still making architectural decisions jointly. Md focused on implementing and validating the ring network, including the route unit, switch unit, and complete network router, and created both directed and randomized unit tests to ensure correct message routing and arbitration. Md also helped debug multicore memory behavior and produced system block diagrams for the report. Jamayne concentrated on composing the full SingleCore and MultiCore system top-levels, connecting the processor, caches, and network interfaces, and developing the single-threaded and multi-threaded sorting microbenchmarks using the `bthread` library. Jamayne also wrote some of the report sections including covering the system design and performance evaluation. Generative AI tools were used for the following ways: (1) troubleshooting syntax and connectivity errors in RTL and C code, (2) suggesting additional randomized testing strategies for stressing multicore memory interactions, and (3) refining the grammar, clarity, and formatting of the final write-up. All key architectural decisions, implementation details, and debugging steps remained fully authored and understood by the team. Additionally, we expanded testing beyond the baseline requirements by designing extra bank-conflict stress tests to further validate correctness under simultaneous shared data-cache access.

8 Appendix

Columns are defined as follows: **Cycles** = total cycles; **Inst** = dynamic instruction count; **CPI** = cycles per instruction; **I-Acc/D-Acc** = instruction/data cache accesses; **I-Miss** = instruction cache misses; **I-MR/D-MR** = instruction/data cache miss rates.

Config	Cycles	Inst	CPI	I-Acc	I-Miss	I-MR	D-Acc	D-MR
Base + ST	4212	812	5.19	913	6	1%	301	25%
Alt + ST	5731	4185	1.37	4805	6	0%	301	25%
Base + MT1	5177	973	5.32	1100	45	4%	355	28%
Alt + MT1	6956	5055	1.38	5812	45	1%	355	24%
Alt + MT4	2678	1617	1.66	1828	64	4%	124	21%

Table 1: Performance metrics for **vvadd**

Config	Cycles	Inst	CPI	I-Acc	I-Miss	I-MR	D-Acc	D-MR
Base + ST	77413	15471	5.00	17267	1708	10%	3637	9%
Alt + ST	95134	73064	1.30	83719	1708	2%	3637	2%
Base + MT1	77367	15424	5.02	17110	2023	12%	3829	8%
Alt + MT1	96613	73295	1.32	83884	2023	2%	3829	3%
Alt + MT4	40659	27421	1.48	30897	2034	7%	1568	6%

Table 2: Performance metrics for **sort**

Config	Cycles	Inst	CPI	I-Acc	I-Miss	I-MR	D-Acc	D-MR
Base + ST	28951	6241	4.64	6800	24	0%	1083	17%
Alt + ST	34573	26305	1.31	29950	24	0%	1083	18%
Base + MT1	28232	6009	4.70	6495	138	2%	1165	18%
Alt + MT1	34002	25952	1.31	29504	138	0%	1165	19%
Alt + MT4	12754	8889	1.43	9801	146	1%	448	22%

Table 3: Performance metrics for **mfilt**

Config	Cycles	Inst	CPI	I-Acc	I-Miss	I-MR	D-Acc	D-MR
Base + ST	12354	1812	6.82	1913	9	0%	801	19%
Alt + ST	16330	11569	1.41	13171	9	0%	801	19%
Base + MT1	13303	1973	6.74	2100	49	2%	855	20%
Alt + MT1	17705	12562	1.41	14317	49	0%	855	19%
Alt + MT4	5405	2605	2.07	2812	70	2%	249	18%

Table 4: Performance metrics for **cmult**

Config	Cycles	Inst	CPI	I-Acc	I-Miss	I-MR	D-Acc	D-MR
Base + ST	10563	2124	4.97	2497	12	0%	240	57%
Alt + ST	11961	9078	1.32	10522	12	0%	240	41%
Base + MT1	11651	2309	5.05	2708	69	3%	302	53%
Alt + MT1	13390	10377	1.29	12018	69	1%	302	40%
Alt + MT4	4375	3083	1.42	3577	99	3%	117	27%

Table 5: Performance metrics for **bsearch**